

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО

ITMO University

Лабораторная работа 5

По дисциплине Администрирование платформ на ОС Linux

Тема работы Управление сервисами systemd, организация резервного копирования и автоматизация хранения данных в S3-хранилище

Обучающийся Рекун Ирина Данииловна

Факультет прикладной информатики

Группа К3220

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся

(дата)

(подпись)

Рекун И.Д.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Береснев А.Д.

(Ф.И.О.)

ВВЕДЕНИЕ

Цель работы: получить практические навыки по управлению ОС Linux с помощью systemd.

Задачи: в ходе выполнения лабораторной работы необходимо:

1. Проанализировать время загрузки системы с использованием systemd-analyze;
2. Получить список активных сервисов и сервисов с автозагрузкой;
3. Исследовать зависимости системных сервисов;
4. Создать собственный service-юнит и проверить его работу через системный журнал;
5. Изучить работу journalctl и фильтрацию записей журнала;
6. Создать пользовательский сервис для nginx;
7. Настроить ограничение памяти с помощью slice-юнита;
8. Создать backend-сервис и настроить порядок запуска компонентов;
9. Автоматизировать задачи с помощью timer-юнитов;
10. Создать target-юнит для управления набором сервисов

На первом этапе лабораторной работы был выполнен анализ процесса загрузки системы. В результате был получен график загрузки системы, позволяющий визуально оценить время запуска ядра и пользовательского пространства, а также последовательность запуска сервисов. Анализ графика показывает, что основное время загрузки приходится на пользовательское пространство (Рисунок 1).

```
root@rekunid:~# systemd-analyze
Startup finished in 5.158s (kernel) + 26.943s (userspace) = 32.101s
graphical.target reached after 26.322s in userspace.
```

Рисунок 1 – Информация о времени загрузки системы

Далее была выполнена команда `systemd-analyze blame`, которая выводит список сервисов по времени их запуска (Рисунок 2). Такой вывод позволяет определить наиболее долгие сервисы и понять, какие компоненты сильнее всего влияют на общее время загрузки системы.

```
8.547s snapd.seeded.service
8.400s cloud-init-local.service
8.236s snapd.service
7.590s postgresql@16-main.service
4.015s dev-sda1.device
3.843s apparmor.service
3.449s systemd-udev-settle.service
2.990s systemd-journal-flush.service
2.876s cloud-init.service
2.679s logrotate.service
2.344s growroot.service
1.781s chrony.service
1.773s networkd-dispatcher.service
1.426s dpkg-db-backup.service
1.336s systemd-networkd-wait-online.service
1.213s rsyslog.service
1.200s cloud-config.service
914ms snapd.apparmor.service
883ms snapd.seeded.service
```

Рисунок 2 – Список сервисов по времени запуска

Для более детального анализа зависимостей между сервисами была использована команда `systemd-analyze critical-chain`. Она позволяет отследить последовательность запуска сервисов и выявить цепочку, влияющую на достижение целевого состояния систем (Рисунок 3).

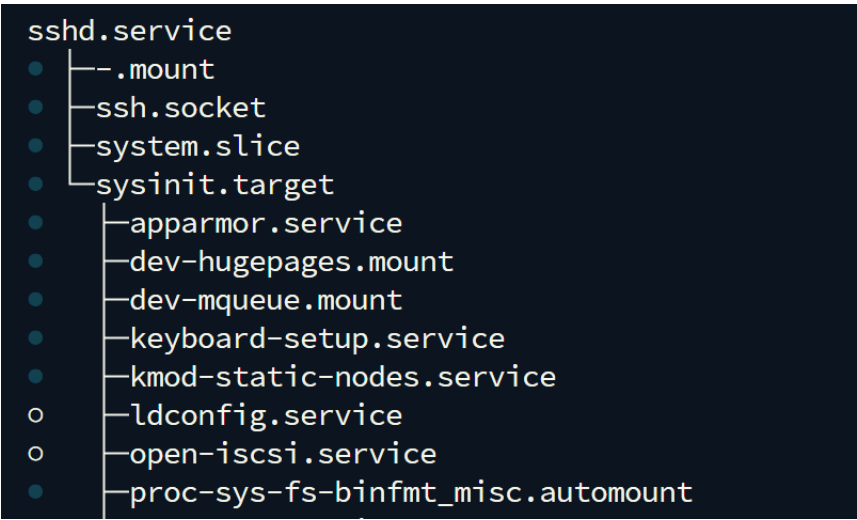


Рисунок 3 – Цепочка загрузки сервисов

После был построен графический отчет загрузки системы. Данный график наглядно отображает временную шкалу запуска всех сервисов и позволяет визуально определить узкие места и задержки в процессе загрузки системы (Рисунок 4).

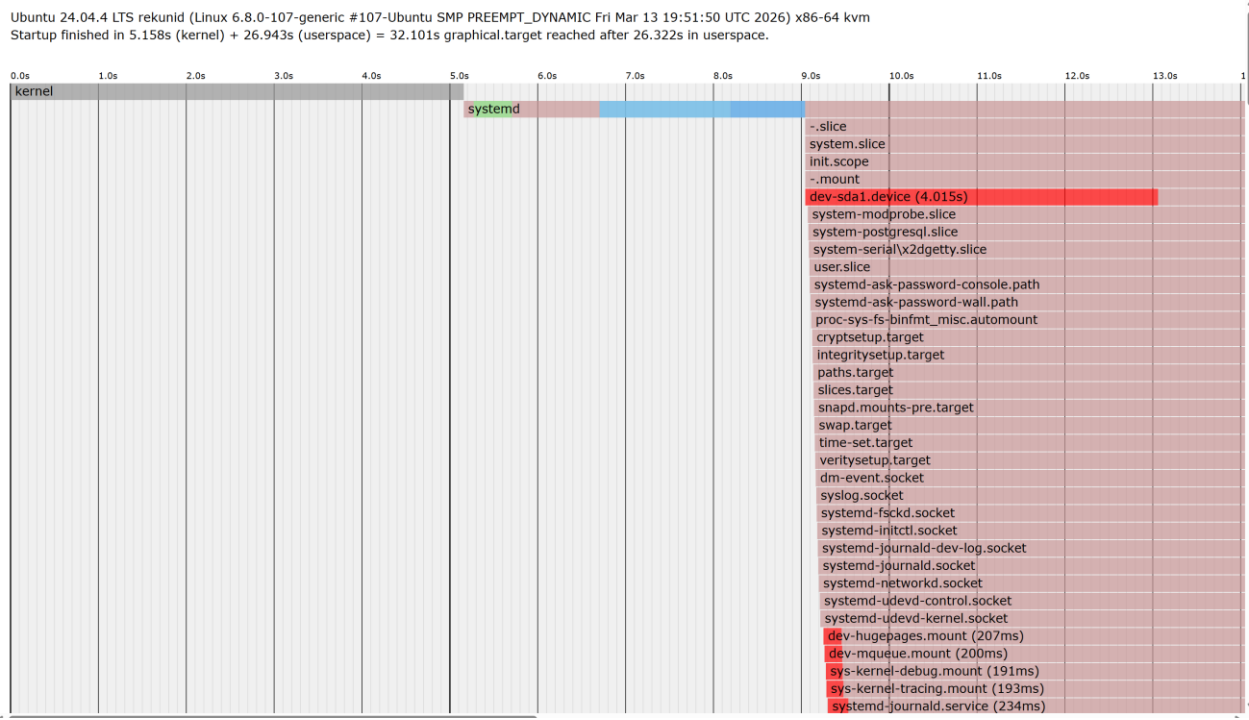


Рисунок 4 – Начало графика загрузки системы

Также был выполнен просмотр списка активных сервисов (Рисунок 5).

```
root@rekunid:~# systemctl list-units --type=service
```

UNIT	LOAD	ACTIVE	SUB	DESCRIPTION
apparmor.service	loaded	active	exited	Load AppArmor profiles
atd.service	loaded	active	running	Deferred execution scheduler
chrony.service	loaded	active	running	chrony, an NTP client/server
cloud-config.service	loaded	active	exited	Apply the settings specified in cloud-config
cloud-final.service	loaded	active	exited	Execute cloud user/final scripts
cloud-init-local.service	loaded	active	exited	Initial cloud-init job (pre-networking)
cloud-init.service	loaded	active	exited	Initial cloud-init job (metadata service crawler)
console-setup.service	loaded	active	exited	Set console font and keymap

Рисунок 5 – Список активных сервисов системы

Для анализа состояния сервисов и их автозапуска была использована команда `systemctl list-unit-files --type=service`. Она позволяет определить, какие сервисы включены в автозагрузку системы, а какие отключены (Рисунок 6).

```
nginx.service
open-iscsi.service
pg_receivewal@.service
postgresql.service
postgresql@.service
rsync.service
rsyslog.service
screen-cleanup.service
serial-getty@.service
set-root-pw.service
setvtrgb.service
snapd.apparmor.service
snapd.autoimport.service
snapd.core-fixup.service
snapd.recovery-chooser-trigger.service
snapd.seeded.service
snapd.service
```

enabled	enabled
enabled	enabled
disabled	enabled
enabled	enabled
indirect	enabled
disabled	enabled
enabled	enabled
masked	enabled
indirect	enabled
enabled	enabled
enabled	enabled
enabled	enabled
enabled	enabled
enabled	enabled
enabled	enabled
enabled	enabled
enabled	enabled

Рисунок 6 – Список сервисов и их состояние автозапуска

Далее был выполнен анализ зависимостей службы `sshd` (Рисунок 7).

```
root@rekunid:~# systemctl list-dependencies sshd
```

```
sshd.service
├─ .mount
├─ ssh.socket
├─ system.slice
├─ sysinit.target
├─ apparmor.service
├─ dev-hugepages.mount
├─ dev-mqueue.mount
├─ keyboard-setup.service
├─ kmod-static-nodes.service
├─ ldconfig.service
├─ open-iscsi.service
├─ proc-sys-fs-binfmt_misc.automount
├─ setvtrgb.service
├─ sys-fs-fuse-connections.mount
├─ sys-kernel-config.mount
├─ sys-kernel-debug.mount
└─ sys-kernel-tracing.mount
```

Рисунок 7 – Зависимости службы `sshd`

Аналогичным образом были проанализированы зависимости веб-сервера `nginx` (Рисунок 8).

```

root@rekunid:~# systemctl list-dependencies nginx
nginx.service
├─system.slice
├─network-online.target
│   └─systemd-networkd-wait-online.service
├─sysinit.target
│   ├──apparmor.service
│   ├──dev-hugepages.mount
│   ├──dev-mqueue.mount
│   ├──keyboard-setup.service
│   ├──kmod-static-nodes.service
│   ○ ldconfig.service
│   ○ open-iscsi.service
│   └─proc-sys-fs-binfmt_misc.automount

```

Рисунок 8 – Зависимости службы nginx

Также был проведен анализ зависимостей службы базы данных PostgreSQL (Рисунок 9).

```

root@rekunid:~# systemctl list-dependencies postgresql
postgresql.service
├─postgresql@16-main.service
├─system.slice
├─sysinit.target
│   ├──apparmor.service
│   ├──dev-hugepages.mount
│   ├──dev-mqueue.mount
│   ├──keyboard-setup.service
│   ├──kmod-static-nodes.service
│   ○ ldconfig.service
│   ○ open-iscsi.service
│   ├──proc-sys-fs-binfmt_misc.automount
│   ├──setvtrgb.service
│   ├──sys-fs-fuse-connections.mount
│   ├──sys-kernel-config.mount
│   ├──sys-kernel-debug.mount
│   ├──sys-kernel-tracing.mount
│   ├──systemd-ask-password-console.path
│   ├──systemd-binfmt.service
│   ○ systemd-firstboot.service
│   ○ systemd-hwdb-update.service

```

Рисунок 9 – Зависимости службы PostgreSQL

Далее был создан отдельный системный пользователь rekunuser, предназначенный для запуска пользовательского сервиса без возможности входа в систему. Для этого была использована команда useradd с указанием оболочки /usr/sbin/nologin (Рисунок 10).

```

root@rekunid:~# sudo useradd -r -s /usr/sbin/nologin rekunuser
root@rekunid:~# sudo nano /etc/systemd/system/rekun.service

```

Рисунок 10 — Создание системного пользователя и файла сервиса

Также создан собственный unit-файл systemd rekun.service, в котором описана служба, выполняющая одноразовую команду при запуске системы (Рисунок 11).

```
[Unit]
Description=Rekun startup message service
After=network.target

[Service]
Type=oneshot
User=rekuser
ExecStart=/usr/bin/logger "Rekun service started"

[Install]
WantedBy=multi-user.target
```

Рисунок 11 — Содержимое unit-файла rekun.service

После создания unit-файла была выполнена перезагрузка конфигурации с помощью команды `systemctl daemon-reload`, а также включение сервиса в автозагрузку (Рисунок 12).

```
root@rekunid:~# sudo systemctl daemon-reload
root@rekunid:~# sudo systemctl enable rekun
Created symlink /etc/systemd/system/multi-user.target.wants/rekun.service → /etc/systemd/system/rekun.service.
```

Рисунок 12 — Активация и добавление сервиса в автозагрузку

Для проверки корректности работы сервиса был выполнен просмотр системного журнала, где видно, что сервис успешно запускается и записывает сообщение (Рисунок 13).

```
Apr 27 15:20:32 rekunid systemd[1]: Starting rekun.service - Rekun startup message service...
Apr 27 15:20:32 rekunid rekunuser[2006]: Rekun service started at $(date)
Apr 27 15:20:32 rekunid systemd[1]: rekun.service: Deactivated successfully.
Apr 27 15:20:32 rekunid systemd[1]: Finished rekun.service - Rekun startup message service.
Apr 27 15:24:48 rekunid systemd[1]: Starting rekun.service - Rekun startup message service...
Apr 27 15:24:48 rekunid rekunuser[2096]: Rekun service started at $(date)
Apr 27 15:24:48 rekunid systemd[1]: rekun.service: Deactivated successfully.
Apr 27 15:24:48 rekunid systemd[1]: Finished rekun.service - Rekun startup message service.
Apr 27 15:28:17 rekunid systemd[1]: Starting rekun.service - Rekun startup message service...
Apr 27 15:28:17 rekunid rekunuser[2168]: Rekun service started
Apr 27 15:28:17 rekunid systemd[1]: rekun.service: Deactivated successfully.
Apr 27 15:28:17 rekunid systemd[1]: Finished rekun.service - Rekun startup message service.
```

Рисунок 13 — Результат работы пользовательского сервиса в журнале

В четвертой части лабораторной работы был выполнен анализ системного журнала с использованием команды `journalctl` (Рисунок 14).

```
root@rekunid:~# journalctl
Feb 24 16:02:22 ubuntu kernel: Linux version 6.8.0-101-generic (build@lcy02-amd64-051) (x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2-
Feb 24 16:02:22 ubuntu kernel: Command line: BOOT_IMAGE=/boot/vmlinuz-6.8.0-101-generic root=LABEL=cloudimg-rootfs ro consoleblank=0 pan
Feb 24 16:02:22 ubuntu kernel: KERNEL supported cpus:
Feb 24 16:02:22 ubuntu kernel: Intel GenuineIntel
Feb 24 16:02:22 ubuntu kernel: AMD AuthenticAMD
Feb 24 16:02:22 ubuntu kernel: Hygon HygonGenuine
Feb 24 16:02:22 ubuntu kernel: Centaur CentaurHauls
Feb 24 16:02:22 ubuntu kernel: zhaoxin Shanghai
Feb 24 16:02:22 ubuntu kernel: BIOS-provided physical RAM map:
```

Рисунок 14 — Просмотр системного журнала

Также в журнале были обнаружены записи о работе ранее созданного сервиса `rekun.service` (Рисунок 15).


```

Apr 27 15:20:32 rekunid systemd[1]: Starting rekun.service - Rekun startup message service...
Apr 27 15:20:32 rekunid rekunuser[2006]: Rekun service started at $(date)
Apr 27 15:20:32 rekunid systemd[1]: rekun.service: Deactivated successfully.
Apr 27 15:20:32 rekunid systemd[1]: Finished rekun.service - Rekun startup message service.
Apr 27 15:24:48 rekunid systemd[1]: Starting rekun.service - Rekun startup message service...
Apr 27 15:24:48 rekunid rekunuser[2096]: Rekun service started at $(date)
Apr 27 15:24:48 rekunid systemd[1]: rekun.service: Deactivated successfully.
Apr 27 15:24:48 rekunid systemd[1]: Finished rekun.service - Rekun startup message service.
Apr 27 15:28:17 rekunid systemd[1]: Starting rekun.service - Rekun startup message service...
Apr 27 15:28:17 rekunid rekunuser[2168]: Rekun service started
Apr 27 15:28:17 rekunid systemd[1]: rekun.service: Deactivated successfully.
Apr 27 15:28:17 rekunid systemd[1]: Finished rekun.service - Rekun startup message service.

```

Рисунок 15 — Логи работы пользовательского сервиса

Кроме того, в журнале присутствуют сообщения об ошибках, например, связанные с SSH и отсутствием некоторых модулей ядра (Рисунок 16).

```

Apr 15 06:54:05 rekunid sshd[16665]: error: kex_protocol_error: type 30 seq 1 [preauth]
Apr 15 06:54:05 rekunid sshd[16665]: error: kex_protocol_error: type 20 seq 2 [preauth]
Apr 15 06:54:05 rekunid sshd[16665]: error: kex_protocol_error: type 30 seq 3 [preauth]
-- Boot 6a41eac7ecc245e9be7552bc790bd87c --
Apr 15 13:20:39 rekunid systemd-modules-load[261]: Failed to find module 'i6300esb'
-- Boot 88f0bb5af283401abe7ce8578c4a8bf2 --
Apr 15 17:19:15 rekunid systemd-modules-load[255]: Failed to find module 'i6300esb'
-- Boot 980ab8655bea4adeb4076a88cf2b2e2 --
Apr 27 14:46:32 rekunid systemd-modules-load[263]: Failed to find module 'i6300esb'
Apr 27 14:46:50 rekunid systemd[1]: Failed to start logrotate.service - Rotate log files.
lines 50-95/95 (END)

```

Рисунок 16 — Примеры ошибок в системном журнале

Для оценки занимаемого журналами объема была выполнена команда, результат которой представлен на рисунке 17. Она позволяет определить, сколько места занимают активные и архивные логи (Рисунок 17).

```

root@rekunid:~# journalctl --disk-usage
Archived and active journals take up 79.1M in the file system.

```

Рисунок 17 — Информация о размере журналов

В заключительной части лабораторной работы было выполнено создание собственного unit-файла на основе nginx. Для этого исходный файл был скопирован и переименован, после чего оригинальный сервис был отключен. Далее был отредактирован unit-файл нового сервиса nginx-custom.service. В конфигурации были заданы параметры запуска (Рисунок 18)

```

[Service]
Type=forking
PIDFile=/run/nginx.pid
ExecStartPre=/usr/sbin/nginx -t -q -g 'daemon on; master_process on;'
ExecStart=/usr/sbin/nginx -c /etc/nginx/nginx.conf -g 'daemon on; master_process on;'
ExecReload=/usr/sbin/nginx -g 'daemon on; master_process on;' -s reload
ExecStop=/sbin/start-stop-daemon --quiet --stop --retry QUIT/5 --pidfile /run/nginx.pid
TimeoutStopSec=5
KillMode=mixed

```

Рисунок 18 – Конфигурация кастомного сервиса nginx

После настройки сервис был перезапущен и проверен его статус с помощью команды `systemctl status`. В выводе видно, что сервис успешно запущен и работает корректно (Рисунок 19).

```
root@rekunid:~# sudo systemctl restart nginx-custom
root@rekunid:~# systemctl status nginx-custom
● nginx-custom.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/etc/systemd/system/nginx-custom.service; disabled; preset: enabled)
   Active: active (running) since Mon 2026-04-27 15:42:46 UTC; 7s ago
     Docs: man:nginx(8)
  Process: 2534 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
  Process: 2536 ExecStart=/usr/sbin/nginx -c /etc/nginx/nginx.conf -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
 Main PID: 2537 (nginx)
    Tasks: 3 (limit: 4660)
   Memory: 2.9M (peak: 3.1M)
      CPU: 23ms
   CGroup: /system.slice/nginx-custom.service
           └─2537 "nginx: master process /usr/sbin/nginx -c /etc/nginx/nginx.conf -g daemon on; master_process on;"
             └─2538 "nginx: worker process"
               └─2539 "nginx: worker process"

Apr 27 15:42:46 rekunid systemd[1]: Starting nginx-custom.service - A high performance web server and a reverse proxy server...
Apr 27 15:42:46 rekunid systemd[1]: Started nginx-custom.service - A high performance web server and a reverse proxy server.
```

Рисунок 19 — Конфигурация кастомного сервиса nginx

В рамках дальнейшей настройки системы было выполнено ограничение использования оперативной памяти для сервиса nginx. Для этого был создан отдельный slice `nginx.slice`, в котором задан параметр `MemoryMax`, ограничивающий максимальное потребление памяти на уровне 30% от общего объема. (Рисунок 20).

```
GNU nano 7.2
[slice]
MemoryMax=30%
```

Рисунок 20 — Ограничение использования памяти для nginx через slice

Далее в unit-файле сервиса `nginx-custom.service` был указан параметр `Slice=nginx.slice`, что привязывает сервис к ранее созданному ограничению. (Рисунок 21).

```
[Service]
Type=forking
PIDFile=/run/nginx.pid
ExecStartPre=/usr/sbin/nginx -t -q -g 'daemon on; master_process on;'
ExecStart=/usr/sbin/nginx -c /etc/nginx/nginx.conf -g 'daemon on; master_process on;'
ExecReload=/usr/sbin/nginx -g 'daemon on; master_process on;' -s reload
ExecStop=-/sbin/start-stop-daemon --quiet --stop --retry QUIT/5 --pidfile /run/nginx.pid
TimeoutStopSec=5
KillMode=mixed
Slice=nginx.slice
```

Рисунок 21 — Применение ограничения памяти к сервису nginx

После перезапуска сервиса была выполнена проверка его состояния, а также значения параметра `MemoryMax` с помощью команды `systemctl show`. В выводе видно, что ограничение успешно применено, что подтверждает корректность настройки (Рисунок 22).

```

root@rekunid:~# sudo systemctl daemon-reload
root@rekunid:~# sudo systemctl restart nginx-custom
root@rekunid:~# systemctl status nginx-custom
● nginx-custom.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/etc/systemd/system/nginx-custom.service; disabled; preset: enabled)
   Active: active (running) since Mon 2026-04-27 15:46:38 UTC; 9s ago
     Docs: man:nginx(8)
  Process: 2621 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
  Process: 2624 ExecStart=/usr/sbin/nginx -c /etc/nginx/nginx.conf -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
 Main PID: 2625 (nginx)
    Tasks: 3 (limit: 4660)
   Memory: 2.9M (peak: 3.2M)
      CPU: 26ms
   CGroup: /nginx.slice/nginx-custom.service
           └─2625 "nginx: master process /usr/sbin/nginx -c /etc/nginx/nginx.conf -g daemon on; master_process on;"
             └─2626 "nginx: worker process"
               └─2627 "nginx: worker process"

Apr 27 15:46:38 rekunid systemd[1]: Starting nginx-custom.service - A high performance web server and a reverse proxy
Apr 27 15:46:38 rekunid systemd[1]: Started nginx-custom.service - A high performance web server and a reverse proxy
root@rekunid:~# systemctl show nginx.slice -p MemoryMax
MemoryMax=1231831040

```

Рисунок 22 — Проверка применения ограничения памяти

Для обеспечения корректного порядка запуска сервисов была добавлена зависимость веб-сервера nginx от базы данных PostgreSQL. В unit-файле были указаны параметры `After=postgresql.service` и `Requires=postgresql.service` (Рисунок 23)

```

[Unit]
Description=A high performance web server and a reverse proxy server
Documentation=man:nginx(8)
After=network-online.target remote-fs.target nss-lookup.target
Wants=network-online.target
ConditionFileIsExecutable=/usr/sbin/nginx
After=postgresql.service
Requires=postgresql.service

```

Рисунок 23 — Добавление зависимости nginx от PostgreSQL

После внесения изменений была выполнена перезагрузка конфигурации systemd и проверка зависимостей. В выводе команды видно, что сервис nginx-custom.service теперь зависит от postgresql.service (Рисунок 24).

```

root@rekunid:~# sudo systemctl daemon-reload
root@rekunid:~# sudo systemctl restart nginx-custom
root@rekunid:~# systemctl list-dependencies nginx-custom
nginx-custom.service
├─nginx.slice
└─postgresql.service

```

Рисунок 24 — Проверка зависимостей nginx-custom

Далее был реализован простой backend-сервис на Node.js, который запускает HTTP-сервер и возвращает текстовый ответ. Сервер настроен на прослушивание адреса 127.0.0.1, что ограничивает доступ к нему только с локальной машины и повышает безопасность системы (Рисунок 25).

```
const http = require('http');

const port = process.env.PORT || 3000;

http.createServer((req, res) => {
  res.end('Backend is running\n');
}).listen(port, '127.0.0.1');
```

Рисунок 25 — Реализация backend-сервиса на Node.js

Для управления backend-приложением был создан unit-файл backend.service, в котором указана команда запуска приложения, а также параметр Restart=always, обеспечивающий автоматический перезапуск при сбоях

(Рисунок 26).

```
GNU nano 7.2
[Unit]
Description=Backend service

[Service]
ExecStart=/usr/bin/node /root/backend.js
Restart=always

[Install]
WantedBy=multi-user.target
```

Рисунок 26 — Конфигурация backend-сервиса

После активации сервиса и его запуска была выполнена проверка его состояния (Рисунок 27).

```
root@rekunid:~# sudo systemctl daemon-reload
root@rekunid:~# sudo systemctl enable --now backend
Created symlink /etc/systemd/system/multi-user.target.wants/backend.service → /etc/systemd/system/backend.service
root@rekunid:~# systemctl status backend
● backend.service - Backend service
   Loaded: loaded (/etc/systemd/system/backend.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-04-27 15:54:27 UTC; 7s ago
     Main PID: 3327 (node)
        Tasks: 10 (limit: 4660)
      Memory: 28.9M (peak: 29.0M)
         CPU: 136ms
       CGroup: /system.slice/backend.service
               └─3327 /usr/bin/node /root/backend.js

Apr 27 15:54:27 rekunid systemd[1]: Started backend.service - Backend service.
```

Рисунок 27 — Проверка работы backend-сервиса

Для корректного взаимодействия компонентов была добавлена зависимость nginx от backend-сервиса (Рисунок 28).

```
[Unit]
Description=A high performance web server and a reverse proxy server
After=postgresql.service backend.service
Requires=postgresql.service backend.service
Documentation=man:nginx(8)
After=network-online.target remote-fs.target nss-lookup.target
Wants=network-online.target
ConditionFileIsExecutable=/usr/sbin/nginx
After=postgresql.service
Requires=postgresql.service
```

Рисунок 28 — Добавление зависимости nginx от backend

Проверка зависимостей показала, что сервис nginx-custom теперь зависит от backend, PostgreSQL и сетевой подсистемы. Это обеспечивает правильный порядок запуска всех компонентов системы (Рисунок 29).

```
root@rekunid:~# systemctl list-dependencies nginx-custom
nginx-custom.service
├─backend.service
├─nginx.slice
├─postgresql.service
├─network-online.target
├─┬─systemd-networkd-wait-online.service
└─┴─sysinit.target
```

Рисунок 29 — Итоговая структура зависимостей сервисов

Для автоматической отправки логов nginx в S3-хранилище был создан сервис nginx-logs-s3.service, выполняющий скрипт загрузки. Сервис имеет тип oneshot, так как выполняется периодически и завершает работу после выполнения задачи (Рисунок 30).

```
[Unit]
Description=Upload nginx logs to S3

[Service]
Type=oneshot
ExecStart=/root/backup.sh
```

Рисунок 30 — Сервис отправки логов в S3

Для регулярного запуска данного сервиса был создан таймер nginx-logs-s3.timer, в котором задано расписание выполнения (Рисунок 31).

```
[Unit]
Description=Run nginx logs upload to S3

[Timer]
OnCalendar=Mon,Wed,Fri 22:00
Persistent=true

[Install]
WantedBy=timers.target
```

Рисунок 31 — Настройка таймера для отправки логов

После активации таймера была выполнена проверка его состояния. В выводе видно, что таймер успешно работает и ожидает следующего запуска (Рисунок 32).

```
root@rekunid:~# sudo systemctl daemon-reload
root@rekunid:~# sudo systemctl enable --now nginx-logs-s3.timer
Created symlink /etc/systemd/system/timers.target.wants/nginx-logs-s3.timer → /etc/systemd/system/nginx-logs-s3.timer.
root@rekunid:~# systemctl status nginx-logs-s3.timer
● nginx-logs-s3.timer - Run nginx logs upload to S3
   Loaded: loaded (/etc/systemd/system/nginx-logs-s3.timer; enabled; preset: enabled)
   Active: active (waiting) since Mon 2026-04-27 15:59:17 UTC; 7s ago
     Trigger: Mon 2026-04-27 22:00:00 UTC; 6h left
    Triggers: ● nginx-logs-s3.service

Apr 27 15:59:17 rekunid systemd[1]: Started nginx-logs-s3.timer - Run nginx logs upload to S3.
root@rekunid:~# systemctl list-timers | grep nginx
Mon 2026-04-27 22:00:00 UTC    6h -    - nginx-logs-s3.timer    nginx-logs-s3
```

Рисунок 32 — Проверка работы таймера

Для организации резервного копирования базы данных был создан bash-скрипт. Это позволяет уменьшить размер резервных копий и упростить их хранение (Рисунок 33).

```
#!/bin/bash
mkdir -p /var/backups/postgres
pg_dumpall | gzip > /var/backups/postgres/backup_$(date +%F_%H-%M).sql.gz
```

Рисунок 33 — Скрипт резервного копирования PostgreSQL

Далее был создан сервис postgres-backup.service, который запускает данный скрипт и автоматизирует процесс создания резервных копий (Рисунок 34).

```
root@rekunid:~# sudo nano /root/pg_backup.sh
root@rekunid:~# sudo chmod +x /root/pg_backup.sh
```

Рисунок 34 — Сервис резервного копирования PostgreSQL

Для регулярного выполнения резервного копирования был настроен таймер postgres-backup.timer, который запускает сервис по расписанию. Это обеспечивает автоматическое создание актуальных копий базы данных (Рисунок 35).

```
Unit]
Description=PostgreSQL backup

[Service]
Type=oneshot
ExecStart=/root/pg_backup.sh
```

Рисунок 35 — Таймер резервного копирования PostgreSQL

После активации таймера была выполнена проверка его состояния с помощью команды `systemctl status`, а также просмотр списка таймеров. В выводе видно, что таймер успешно зарегистрирован и ожидает следующего запуска (Рисунок 36).

```

root@rekunid:~# sudo systemctl enable --now postgres-backup.timer
Created symlink /etc/systemd/system/timers.target.wants/postgres-backup.timer → /etc/systemd/system/postgres-backup.timer.
root@rekunid:~# systemctl status postgres-backup.timer
● postgres-backup.timer - Run PostgreSQL backup
   Loaded: loaded (/etc/systemd/system/postgres-backup.timer; enabled; preset: enabled)
   Active: active (waiting) since Mon 2026-04-27 16:06:18 UTC; 13s ago
   Trigger: Tue 2026-04-28 22:00:00 UTC; 1 day 5h left
   Triggers: ● postgres-backup.service

Apr 27 16:06:18 rekunid systemd[1]: Started postgres-backup.timer - Run PostgreSQL backup.
root@rekunid:~# systemctl list-timers | grep postgres
Tue 2026-04-28 22:00:00 UTC 1 day 5h - - postgres-backup.timer postgres-backup.servi

```

Рисунок 36 — Проверка работы таймера резервного копирования

В завершение была создана агрегирующая служба, объединяющая все основные компоненты системы: веб-сервер nginx, базу данных postgresql и backend-приложение. В unit-файле заданы параметры Requires и After, что обеспечивает корректный порядок запуска всех сервисов и позволяет запускать систему как единый стек (Рисунок 37).

```

[Unit]
Description=My application stack
Requires=nginx-custom.service postgresql.service backend.service
After=nginx-custom.service postgresql.service backend.service

```

Рисунок 37 — Итоговый unit-файл системы

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы были изучены механизмы управления сервисами в Linux с помощью systemd, выполнена настройка и анализ зависимостей служб, реализовано ограничение ресурсов. Также была установлена и настроена СУБД PostgreSQL, организовано автоматическое резервное копирование и настроена отправка данных в S3-хранилище. Полученные навыки позволяют эффективно администрировать сервисы и обеспечивать надежность работы системы.